

FTRIA-120 — Runtime Invariant Trigger Algebra

Part II — Runtime Invariant Algebra

Sizhe Tan

with chatGPT (OpenAI)

2026-07-15

Repository: <https://github.com/sizhet/Function-Tunnel-and-Runtime-Invariant-Algebra-FTRIA>

Abstract

Discovering a Runtime Invariant does not execute it.

Extending a Runtime Invariant does not activate it.

Intelligent systems ultimately require mechanisms that determine **when**, **where**, and **how** Runtime Invariants participate in runtime computation.

This paper introduces **Runtime Invariant Trigger Algebra (RITA)** as the third major branch of Runtime Invariant Algebra.

Rather than viewing triggering as isolated control flow or execution logic, RITA interprets triggering as the activation of Runtime Invariants under evolving runtime conditions.

This perspective unifies Common Concept Core (CCC) Triggering, Calling Graph execution, Function Tunnel traversal, Transformer Attention, Agent planning, scheduling, and many existing runtime mechanisms within a common algebraic framework.

1. From Extension to Runtime Activation

Discovery identifies Runtime Invariants.

Extension enlarges their admissible realization space.

Triggering determines when a Runtime Invariant becomes operational.

Conceptually,

Runtime Invariant Discovery

↓

Runtime Invariant Extension

↓

Runtime Invariant Triggering



Runtime Execution

Without Triggering,
Runtime Invariants remain latent structural assets.

2. What Is Runtime Invariant Triggering?

A Runtime Invariant Trigger is a runtime operation that activates an existing Runtime Invariant under appropriate structural and contextual conditions.

Typical triggering decisions include

- selecting,
- activating,
- invoking,
- scheduling,
- routing,
- composing,
- sequencing,
- coordinating.

Unlike Discovery,

Triggering does not search for a new Runtime Invariant.

Unlike Extension,

Triggering does not enlarge its Configuration Space.

Instead,

Triggering determines which Runtime Invariant should participate in the current runtime state.

3. Runtime Invariant Trigger Operators

Many existing systems naturally perform Runtime Invariant Triggering.

Examples include

Structural Intelligence

- CCC Trigger,
- Calling Graph execution,
- Function Tunnel traversal,
- Runtime dispatch.

Software Engineering

- function invocation,
- scheduler decisions,

- workflow execution,
- event dispatch,
- dependency activation.

Modern AI

- Transformer Attention,
- Agent planning,
- memory retrieval,
- policy selection,
- retrieval-augmented generation (RAG),
- tool invocation.

Although implemented differently,

these mechanisms share a common Runtime Function:

Activate appropriate Runtime Invariants according to the current runtime context.

4. Runtime Context Determines Triggering

Triggering is inherently context dependent.

The same Runtime Invariant may be

- activated,
- postponed,
- combined,
- bypassed,
- or suppressed,

depending upon

- runtime state,
- execution history,
- environmental constraints,
- available resources,
- task objectives.

Consequently,

Triggering is not static execution.

It is context-sensitive Runtime Invariant activation.

5. Triggering as Navigation Within Configuration Space

Part I established that every Runtime Invariant possesses a Configuration Space.

Triggering may therefore be interpreted geometrically.

Instead of selecting isolated procedures,

Triggering navigates among admissible Runtime Invariants.

Conceptually,

Runtime Context



Candidate Runtime Invariants



Runtime Trigger



Activated Runtime Invariant



Runtime Evolution

The Trigger selects a feasible trajectory rather than merely executing predefined instructions.

6. Explicit and Implicit Triggering

Runtime Invariant Triggering may occur explicitly or implicitly.

Explicit Triggering

Examples include

- CCC Trigger,
- Calling Graph dispatch,
- workflow engines,
- event-driven execution.

Implicit Triggering

Examples include

- Transformer Attention,
- policy networks,
- adaptive scheduling,
- neural routing,
- dynamic memory retrieval.

Different computational mechanisms,

yet the same Runtime Function:

activating Runtime Invariants under runtime conditions.

7. Runtime Invariant Trigger Algebra

Runtime Invariant Trigger Algebra provides a common language for activation mechanisms.

Instead of describing execution by implementation,

RITA classifies operations according to their structural effect on Runtime Invariants.

Typical Trigger Operators include

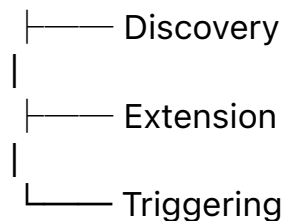
- activate,
- select,
- compose,
- route,
- schedule,
- synchronize,
- coordinate,
- sequence.

Each operator determines how Runtime Invariants participate in runtime evolution.

8. Relationship to Discovery and Extension

Runtime Invariant Algebra consists of three complementary branches.

Runtime Invariant Algebra



Discovery identifies Runtime Invariants.

Extension expands their admissible Configuration Spaces.

Triggering activates Runtime Invariants during execution.

Together,

these three branches describe the complete lifecycle of Runtime Invariants, from identification, through evolution, to runtime realization.

Key Takeaways

- Runtime Invariant Triggering activates existing Runtime Invariants rather than discovering or extending them.

- Triggering is determined by runtime context.
- Many existing mechanisms—including CCC Trigger, Calling Graph execution, Transformer Attention, Agent planning, and workflow scheduling—share the same Runtime Function.
- Triggering may occur explicitly or implicitly.
- Runtime Invariant Trigger Algebra provides a unified interpretation of runtime activation across software engineering and modern AI.
- Discovery, Extension, and Triggering together form the three fundamental branches of Runtime Invariant Algebra.

Related FTRIA Documents

Discovery Algebra

- FTRIA-101 — Runtime Invariant Discovery Algebra
- FTRIA-102 — Common Concept Core (CCC) as Runtime Invariant Discovery
- FTRIA-106 — LLM Word Embeddings as Approximate Runtime Invariant Encoding
- FTRIA-107 — Transformer Attention as Dynamic Runtime Invariant Discovery

Extension Algebra

- FTRIA-110 — Runtime Invariant Extension Algebra
- FTRIA-111 — CCC Preserve Generation as RI-Preserving Transformation
- FTRIA-116 — Fine-Tuning, LoRA and PEFT as Runtime Invariant Extension

Trigger Algebra

- FTRIA-121 — CCC Trigger as Runtime Invariant Activation
- FTRIA-122 — Calling Graph as Runtime Trigger Network
- FTRIA-123 — Function Tunnel as Runtime Trajectory
- FTRIA-125 — Transformer Attention as Runtime Triggering